

用者可以以缓存命中的方式直接访问。这对于加速被调用者处理请求是有“隐式”提升的。

不过直接进程切换也有一些问题，如可能破坏实时场景下任务的优先级。如果只要调用者发起通信，被调用者就一定响应，那么在一些实时的场景下将无法保证对不同优先级任务的区分处理。L4 的后续系统会在进程间通信的过程中，通过比较调用者和被调用者的优先级来判断是否能够直接调用。

8.5.3 L4 通信连接

L4 系列的微内核经历了直接通信到间接通信的转变。早期 L4 的设计使用线程作为通信的目标，这是为了避免过多的中间抽象带来的缓存和 TLB 污染的性能开销。Liedtke 曾经指出，用类似 Mach 中的端口的抽象，可能会产生 12% 的开销^[2]。

不过，线程作为通信目标的方案也有一些问题^[8]。首先，该模型要求线程 ID 是全局唯一的标识符。全局 ID 在后续的系统工作中被证明会引入潜在的隐蔽信道（Covert Channel）的危险，而这会导致攻击和信息泄露的发生。其次，该模型的信息隐藏性差。多线程服务必须向客户端进程公开其内部结构，比如包含几个线程，每个线程的 ID 是多少，不然就很难进行负载均衡相关的操作。随着现代处理器的能力变得越来越强，并且支持大页等机制，间接通信带来的 TLB 污染问题已经缓和了很多。

这些改变使得后续的系统更倾向于基于信箱的间接通信。例如 seL4 和 Fiasco.OC 最终仍然使用类似 Mach 的方案，将端口作为通信的目标。

8.5.4 L4 通信控制（权限检查）

L4 经历了直接通信到间接通信的转变，其通信控制（或者说权限检查）在这两类通信连接方式下的设计也有所不同。

直接通信下，只要有对应的标识符（如进程号），一个进程就可以尝试和另一个进程通信。在 L4 的早期设计中，内核在通信过程中会将发送者的一个“不可伪造的标识符”传递给接收者，帮助接收者判断是否响应发送者的消息。然而，恶意的发送者进程可能用大量的消息来“轰炸”接收者进程。接收者进程需要花费大量的时间来检查并丢弃恶意消息，这也会影响其执行有用的工作。这样的“轰炸”其实构成了拒绝服务的攻击。

早期 L4 通过氏族和酋长（Clans and Chiefs）^[10]的机制来解决这个问题。具体来说，整个系统内的进程按照“氏族”的层次结构进行组织，每个氏族都有一个指定的“酋长”。在氏族内部，所有消息都是自由传输的。而跨氏族的消息（无论是传出的还是传入的）都将重定向到氏族酋长，由其来控制消息的流向。氏族和酋长机制除了保护内部的进程外，还能够限制不受信任的进程。即如果存在某个不受信任的进程希望将在内部获取的敏感信息传递到外部进程，那么会被酋长检测到并阻止。

氏族和酋长在后续的 L4 系统中逐渐被放弃^[8]。这是因为其有以下问题。首先，一

旦消息需要发到外部，那么中间会经过几次重定向，通过一层层的酋长往外发送。每次这样的重定向相当于增加了两次 IPC 的调用，而这显然是相当大的开销。其次，即使不考虑性能开销，酋长本身仍然是可能被攻击的一个点。恶意的攻击者可以通过攻击酋长来限制整个氏族和外部的通信。

在间接通信的场景下，L4 系列的微内核组件采用更灵活的权限机制 Capability 来解决上述问题。简单来看，Capability 是对于内核对象（可以将微内核中几乎所有的内核结构都抽象成内核对象，包括 IPC、内存等）的索引，以及对于该内核对象的权限。以 IPC 为例，微内核会在内核中为每个通信连接维护一个 IPC 内核对象。这个内核对象中包含接收者、发送者、缓冲区等和通信相关的信息。发送者进程要发起通信时，需要告知内核一个特定的 Capability 来发起通信，内核负责检查 Capability 的正确性及其权限，然后通过 Capability 找到对应的内核对象以及与之对应的接收者，从而开始通信过程。这也就意味着，两个进程间要通信，必须首先通过命名服务等方式获取对应的 Capability，否则它们会在执行通信逻辑时被内核拦截下来，从而避免前文提到的拒绝服务的攻击。

在现代的主流微内核系统中，几乎都会使用 Capability 来管理内核对象并负责通信的权限检查。

练习

8.9 L4 的直接进程切换主要避免了哪部分通信开销？

8.6 案例分析：LRPC 的迁移线程模型

本节主要知识点

- L4 中的直接进程切换和 LRPC 中的迁移线程模型的异同分别是什么？
- LRPC 的数据传递（或消息传递）机制是怎样设计的？

这一节将介绍一种比较“极端”的优化性能的 IPC 设计：迁移线程（Thread Migration）。截至目前，我们了解到优化 IPC 性能的大部分工作关注两个部分：优化控制流切换的性能和优化数据传输的性能。迁移线程认为，可以将其他的 IPC 设计看成将需要处理的数据发送到另一个进程处理，这也是为什么控制流切换和数据传输会成为主要的瓶颈。如果换一个角度，将另一个进程处理数据的代码拉到当前进程，那么我们是不是可以避免控制流的切换（仍然是当前进程处理）以及数据传输（数据已经准备在当前进程中）？迁移线程就是围绕这个新的视角进行设计的。

8.6.1 迁移线程模型

迁移线程方案被用在 LRPC^[11]、Mach (优化版本)^[12] 等系统中,是目前纯软件进程间通信优化中效果最好的设计之一。迁移线程的基本原则是:简化控制流转换,让客户端线程执行“服务端的代码”;简化数据传输,共享参数栈和寄存器;简化接口,优化序列化等开销;优化并发,避免共享的全局数据结构。其中,前两点原则都是基于“将代码拉到本地”这个新的视角。

迁移线程 IPC 和主流 IPC 设计的对比如图 8.11 所示。要做到“将代码拉到本地”,迁移线程首先需要对线程结构进行解耦,明确线程中哪些部分是对通信请求处理起关键作用的。然后,这部分线程允许被调用者(负责处理请求的逻辑)运行在调用者的上下文中,将跨进程调用变成更接近函数调用的形式。

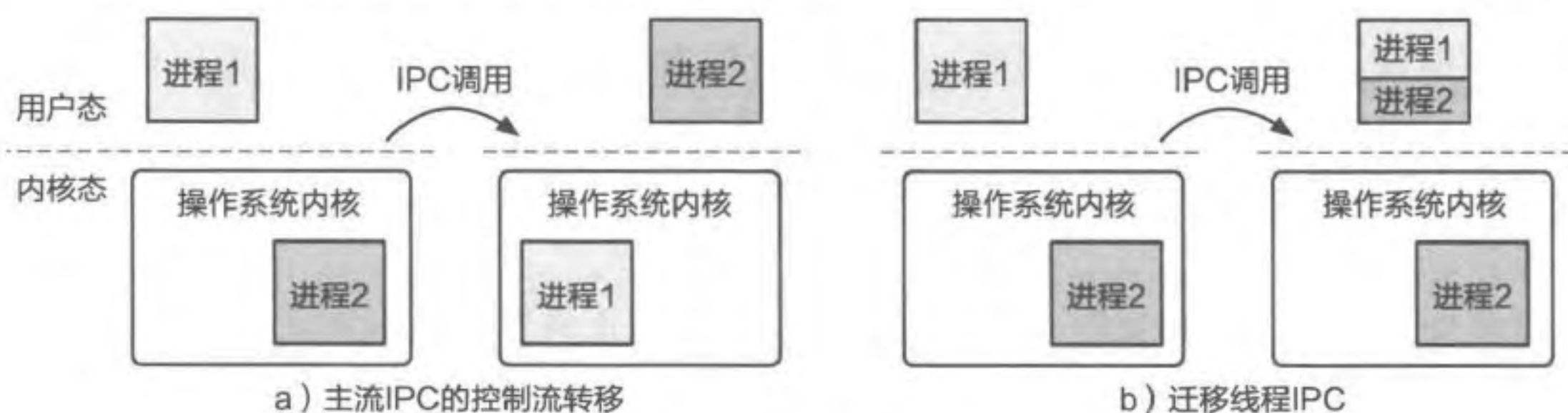


图 8.11 IPC 的设计对比

采用迁移线程模型,在进程间通信过程中,内核不会阻塞调用者线程,但是会让调用者线程执行被调用者的代码。整个过程没有被调用者线程被唤醒,相反,被调用者端更像是一个“代码提供者”。此外,内核不会进行完整的上下文切换,而是只切换地址空间(页表)等和请求处理相关的系统状态。其中,不涉及线程和优先级的切换,也不会调用调度器。迁移线程的优点在于减少了内核调度的时间,并简化了内核中的 IPC 处理。在多核场景下,迁移线程方案还可以避免跨核通信引入的开销。

8.6.2 LRPC 设计

LRPC (Lightweight Remote Procedure Call) 是一种同步的进程间通信设计,客户端通过进程间通信让服务端来执行一个方法(接收来自客户端的参数),并将计算结果返回。此处以 LRPC 为例,展开迁移线程的设计。

共享参数栈和寄存器

首先来看数据传输方面的设计。LRPC 主要通过参数栈 (Argument Stack) 和寄存器来传递数据。顾名思义,参数栈中存放着远程方法调用中客户端向服务端传递的参数。系统内核为每一个 LRPC 连接预先分配好一个参数栈,并将其同时映射在客户端进程和服务端进程地址空间中。因此,在通信的过程中,客户端进程只需要将参数准备到参数栈即可,不需要内核的额外拷贝。这个过程和函数调用中准备参数的过程是类似的。和

L4 中短消息的 IPC 数据通信方式类似，LRPC 在通信调用的过程中不会切换通用寄存器，而是直接使用当前的通用寄存器。

寄存器和参数栈一起构成了 LRPC 的通信设计。客户端进程会优先使用寄存器，在寄存器不够的情况下则用参数栈传递参数。

通信连接的建立

内核需要为通信的服务端提供一个服务的抽象，即服务描述符。所有支持客户端调用的服务端进程将自己的处理函数等信息注册到服务描述符中。在系统内核里，需要为每个服务描述符准备两个资源：第一是参数栈，第二是连接记录（Linkage Record）。参数栈我们已经介绍过了，它将被同时映射到调用者进程和被调用者进程。而连接记录主要是记录调用过程的信息，类似于函数调用中往栈上压入的返回地址等。当一个服务被调用，服务端进程执行完请求需要返回时，内核会从连接记录中获得返回地址等信息。当客户端申请和服务端建立连接时，内核会分配参数栈和连接记录，并返回给客户端进程一个绑定对象（Binding Object），后续客户端可以通过绑定对象发起通信。绑定对象的获得意味着客户端和服务端成功建立了连接。

通信过程

当调用者发起一次通信时：内核验证绑定对象的正确性，并找到正确的服务描述符；内核验证参数栈和连接记录的正确性；内核检查是否有并发调用（可能导致参数栈等异常）；内核将调用者的返回地址和栈指针放到连接记录中；内核将连接记录放到线程控制结构体中的栈上（支持嵌套 LRPC 调用）；内核切换到被调用者进程地址空间；内核找到被调用者进程的运行栈（执行代码所使用的栈）；内核将当前线程的栈指针设置为被调用者进程的运行栈地址；内核将代码指针指向被调用者地址空间中的处理函数。从上面的过程中，我们可以看到 LRPC 设计将页表、运行栈以及代码指针视为负责处理请求的状态。对应的伪代码如代码片段 8.15 所示。

代码片段 8.15 LRPC 的伪代码

```

1 int ipc_call(A_stack, ...)           // A_stack 为参数栈
2 {
3     verify_binding(A_stack);          // 验证参数栈的正确性
4     service_descriptor = get_desc_from_A(A_stack);
5     // 其他安全检查：是否存在并发调用？
6     ...
7     save_ctx_to_linkage_record();      // 将调用信息保存到连接记录上
8     save_linkage_record();
9     ...
10    // 切换运行状态
11    switch_PT();                       // 修改页表
12    switch_cap_table(); // 修改权限表
13    switch_sp();                       // 修改栈地址
14    ...
15    // 返回用户态（服务端进程），不修改参数寄存器
16    ctx_restore_with_args(ret);

```

```
17 ...  
18 }
```

值得注意的是，不同的系统设计中需要考虑的要切换的状态是不同的。例如，代码片段 8.15 中我们将权限表也作为要切换的关键状态之一。

LRPC 中使用的迁移线程模型和 L4 中的直接进程切换有相似的地方，如它们都选择了绕开内核调度。不同点在于，L4 仍然完成了两个线程切换的任务，而在 LRPC 中其实没有切换线程。这在多核场景下的跨核进程间通信中有显著的不同：LRPC 在通信时可以将远端核上的被调用者上下文拉到当前核的线程上直接执行，相当于将跨核通信变成了单核通信；而 L4 却需要核间中断（IPI）等机制来通知远端核进行处理。

微内核进程间通信与宏内核进程间通信

目前介绍的 IPC 设计中既包括宏内核场景下的方案也包括微内核场景下的方案。由于宏内核和微内核的系统设计不同以及对进程间通信的要求不同，其设计通常存在一些差异。

首先，宏内核和微内核在进程设计上的差别会导致 IPC 设计的不同。相比宏内核，微内核架构中的进程既能够表示一个用户态的程序，也可以表示一个系统服务（如文件系统）。这需要微内核 IPC 机制引入用户态的命名服务等机制来管理进程和系统服务间的连接。而宏内核下的命名服务相对简单，是由内核自己完成的，如 Linux 的命名管道依赖文件名来实现命名服务的效果。后续章节将介绍的 Binder IPC 通信机制通常被认为借鉴了微内核的设计，其中一个原因就是 Android 存在大量的用户态系统服务，需要较为复杂的 IPC 管理机制。

其次，宏内核和微内核架构对于 IPC 的性能需求是不一样的，这同样影响了两类架构下的 IPC 设计。相比宏内核架构，微内核架构下的 IPC 频率显著增多，这要求 L4/seL4 等微内核系统采用更精巧的方式来优化 IPC 开销，如在切换上下文过程中只切换一个上下文的“最小集”。

最后，宏内核和微内核架构在内核的复杂性上也影响了各自的 IPC 设计。相比微内核的简洁，宏内核在内核层面包含大量的系统服务、驱动等代码。系统设计者通常很难在这样一个复杂的系统中引入一条 IPC 的快速路径。

8.7 案例分析：ChCore 进程间通信机制

本节主要知识点

□ ChCore 是如何设计和实现进程间通信机制的？